PRODUCT REQUIREMENTS DOCUMENT

Flowchart Generation AI Agent

Enterprise Internal Platform — EY Deployment

Client	Ernst & Young (EY)
Author	[Your Name / Team]
Version	1.0 — Draft
Date	March 2026
Status	For Internal Review
Classification	Confidential

1. Executive Summary

This document defines the product requirements for an AI-powered Flowchart Generation Agent to be deployed as a core feature within an enterprise internal platform built for Ernst & Young (EY). The agent enables EY professionals across all service lines — Consulting, Tax, Assurance & Audit, and Strategy & Transactions — to automatically generate, visualise, and iteratively refine structured flowcharts and workflow diagrams from natural language inputs, uploaded documents, and meeting records.

The agent addresses a widespread operational pain point: consultants and practitioners currently build complex process flows, decision trees, and workflow diagrams manually using tools like Figma, Visio, or PowerPoint — a time-intensive process that requires technical diagramming proficiency and introduces version control challenges. By replacing this manual effort with a conversational AI agent, EY teams can significantly accelerate the production of high-quality diagrams while maintaining the ability to make precise, targeted edits using plain language.

This PRD is intended for the engineering and product team responsible for building the agent. It covers context and motivation, user personas, detailed functional requirements, edge case handling, the UI/UX model, the agentic workflow, integration considerations, and open questions.

2. Background & Context

2.1 About the Deploying Organisation

The platform deploying this agent is built by a company specialising in enterprise AI enablement — specifically the development and on-premise deployment of large language models and AI agent systems for enterprise clients. The core value proposition is providing organisations like EY with AI capabilities that run entirely within their own infrastructure, ensuring data privacy, regulatory compliance, and security without dependence on third-party cloud inference providers.

The flowchart agent is one feature within a broader enterprise AI platform being delivered to EY. The platform is designed to serve multiple departments and use cases under a unified interface, with the flowchart agent representing a high-priority, high-visibility feature given its direct applicability to daily consulting workflows.

2.2 About EY and the Target User Base

EY is one of the Big Four professional services firms, operating across four major service lines globally. Each of these lines relies heavily on structured process documentation, decision frameworks, and workflow diagrams — both for internal operations and for client deliverables.

The agent is intended as an internal capability and will be used by a wide range of professionals including: management consultants building client process maps, tax professionals mapping compliance workflows and decision logic, auditors documenting internal control flows and risk assessments, and deals teams mapping due diligence workflows and integration plans. The user base is not primarily technical — users are highly intelligent domain experts who communicate in business and professional language, not in diagramming syntax or technical terminology.

2.3 The Problem Being Solved

Currently, EY professionals who need to produce flowcharts and workflow diagrams face the following challenges. First, the manual effort is significant — building a moderately complex workflow in Figma or Visio from scratch can take hours, and this time is spent on formatting and layout rather than on thinking. Second, there is a skill gap: not all practitioners are comfortable with diagramming tools, creating dependency on certain individuals or external designers. Third, iteration is painful — when a client or senior stakeholder requests changes to a diagram, re-editing it manually requires re-opening the source file, locating the right element, and carefully adjusting surrounding logic. Fourth, there is poor traceability between the original discussion and the resulting diagram — the translation from spoken or written content to a visual artefact involves significant manual interpretation.

The Flowchart Agent directly addresses all four of these problems by enabling any EY professional to produce a high-quality diagram from a natural language description or uploaded document in minutes, and to refine it further through conversational instruction.

3. Goals & Non-Goals

3.1 Goals

- Enable any EY user to generate a structured, visually clean flowchart from natural language input, uploaded documents, or a combination of both.
- Support a wide range of flowchart types used across EY service lines — process flows, decision trees, data flows, org/governance flows, risk and control flows, RACI diagrams, and more.
- Allow users to iteratively refine generated diagrams through plain language instructions, with changes applied surgically to specific nodes or segments without regenerating the entire diagram.
- Deliver the agent via a split-pane UI: a conversational chat interface on the left and a live diagram canvas on the right, both updating in sync.
- Ensure the system handles ambiguous, incomplete, or high-level input gracefully by asking clarifying questions or making reasonable default assumptions that are transparently communicated.
- Produce diagrams that are exportable to standard formats (PNG, SVG, PDF, and ideally Visio-compatible XML or Mermaid syntax).
- Maintain full conversation and diagram history within a session so users can undo, revisit earlier versions, or branch from a previous state.

3.2 Non-Goals

- This agent is not intended to replace dedicated diagramming tools for highly customised visual design work requiring pixel-perfect control.
- The agent will not perform real-time collaboration (multi-user simultaneous editing) in the initial version.
- The agent is not a document management system — uploaded files are used as input context and are not stored or indexed for retrieval across sessions unless the broader platform provides this capability.
- The agent will not generate numerical models, financial projections, or quantitative analysis — it is focused exclusively on workflow and process visualisation.

4. User Personas

The following personas represent the primary user archetypes across EY service lines. Understanding these personas is critical for calibrating input handling, output style, and the level of assumed domain knowledge.

Persona	Description	Primary Use Case
The Consultant	Mid-to-senior level management consultant working on client engagements. Regularly produces process maps, operating model diagrams, and transformation roadmaps. Tech-comfortable but not a developer. Time-constrained.	Narrates a client workshop outcome and gets an instant process flow. Iterates until it matches what was discussed. Exports for a client deck.
The Tax Professional	Tax advisor or manager handling multi-entity compliance workflows. Needs decision trees to map tax treatment logic. Expects precision and correctness in branching.	Describes a tax determination scenario; the agent builds a decision tree with clearly labelled Yes/No branches.
The Auditor	Assurance professional conducting walkthroughs or process audits. Needs risk-and-control flows and internal process documentation. Familiar with COSO/SOX language.	Uploads an internal audit memo; agent generates a process flow with control points highlighted at each stage.
The Deals Analyst	SaT analyst supporting M&A transactions. Needs to map due diligence workflows, data room processes, or Day 1 integration plans. Works under tight deadlines.	Rapidly generates a multi-phase integration workflow from a deal brief. Adjusts a specific workstream mid-diagram without touching the others.
The Internal Operations Lead	EY internal function (HR, Finance, IT) lead documenting internal processes for compliance, training, or handover purposes.	Documents a procurement approval workflow by describing it verbally; shares output with their team.

5. Supported Flowchart Types

The agent must be capable of generating the following categories of diagrams. Each type has distinct structural conventions, labelling patterns, and node/connector semantics that the underlying model must understand and apply correctly.

5.1 Process Flow Diagrams

The most common type. Depicts a sequence of steps, tasks, or activities from a start state to an end state, with optional parallel paths and loops. Used across all service lines. Conventions include swim lanes for cross-functional processes, clear start/end terminators, task rectangles, and decision diamonds.

- Subtype: Linear process flow (single path, sequential steps)
- Subtype: Cross-functional / swim lane flow (multiple actors or departments)
- Subtype: Circular/iterative process flow (for continuous or cyclical processes)

5.2 Decision Trees

Branching diagrams where each node poses a question or condition and branches lead to different outcomes based on Yes/No or multi-value answers. Heavily used in Tax for tax treatment logic and in Audit for assessment frameworks. The agent must correctly model mutually exclusive branches and ensure all paths terminate.

- Subtype: Binary decision tree (Yes/No at each node)
- Subtype: Multi-branch decision tree (more than two outcomes per node)
- Subtype: Hybrid decision-process tree (decision branches that lead into sub-processes)

5.3 Data Flow Diagrams (DFDs)

Diagrams showing how data moves between systems, processes, people, or external entities. Used heavily in technology consulting engagements, ERP transformation projects, and IT due diligence. Key elements include data stores, processes, external entities, and directional data flows with labelled arrows.

- Subtype: Level 0 / Context diagram (high-level system boundary)
- Subtype: Level 1 / Decomposed DFD (showing internal processes within the system)

5.4 System / Architecture Flow Diagrams

Used to depict how technical systems, applications, or infrastructure components interact. Relevant in technology consulting, IT advisory, and SaT due diligence. Includes application architecture flows, API interaction maps, and microservices dependency diagrams.

5.5 Organisational & Governance Flows

Diagrams depicting reporting structures, decision-making authority, governance hierarchies, or accountability frameworks. Used in operating model design, post-merger integration, and compliance governance.

- Subtype: Org chart / reporting line flow
- Subtype: RACI diagram (Responsible, Accountable, Consulted, Informed)
- Subtype: Governance escalation flow (who escalates to whom under what conditions)

5.6 Risk & Control Flow Diagrams

Core to Assurance and Audit work. These diagrams map a business process and overlay identified risks and the controls designed to mitigate them. Used in SOX walkthroughs, internal audit fieldwork, and risk assessments. Nodes include process steps, risk points (often annotated), and control activities.

5.7 Customer / User Journey Maps

Diagrams depicting a customer's or user's experience across a series of interactions or touchpoints with a product, service, or process. Used in CX consulting, digital transformation, and service design engagements.

5.8 Integration & Migration Workflow Diagrams

Used in M&A and transformation contexts to map how two organisations' processes, systems, or teams will integrate or transition. Includes Day 1 readiness flows, data migration pipelines, and workstream dependency maps.

5.9 Approval & Escalation Flows

Documents how requests, approvals, exceptions, or escalations move through an organisation. Common in Tax, Finance, and HR functions. Typically involves multiple actors, threshold conditions, and time-based triggers.

5.10 Compliance & Regulatory Process Flows

Maps a compliance obligation or regulatory reporting process end-to-end. Used in Tax, Audit, and regulatory advisory contexts. May include statutory deadlines, regulatory body interactions, and penalty/exception paths.

6. Functional Requirements

6.1 Input Handling

The agent must accept and correctly process the following input modalities. Handling each correctly — including gracefully managing ambiguity — is one of the most critical engineering requirements of this feature.

6.1.1 Natural Language Text Input (Chat)

Users type or speak a description of the workflow they want to diagram. The input may be high-level and imprecise (e.g., 'show the process for approving a vendor invoice') or highly detailed with step-by-step breakdowns, conditions, and actors. The agent must parse this into a structured internal representation of nodes, edges, conditions, and actors before generating the diagram.

Edge Cases — Text Input

- Input is too vague: agent must identify what information is missing and ask one clarifying question at a time (not a list of five questions at once).
- Input contains contradictory steps: agent flags the contradiction and asks the user to resolve it before rendering.
- Input describes a process the agent cannot confidently categorise: agent defaults to a simple process flow and informs the user, offering to change the type.

- Input uses highly domain-specific jargon (e.g., SOX, COGS, EBITDA, GST): the agent must understand common EY-domain terminology and not misinterpret it.
- Input is extremely long (e.g., a 2000-word description): agent chunks it, builds the flow section by section, and surfaces a summary of what it understood before rendering.
- Input is in a language other than English: agent detects the language and responds in kind, rendering node labels in the same language.

6.1.2 Document Upload

Users can upload files containing process descriptions, meeting notes, project briefs, or similar documents. The agent must extract relevant workflow content, ignore boilerplate or irrelevant sections, and construct a diagram from the extracted logic.

Supported file formats must include: PDF, DOCX, PPTX, TXT, and ideally CSV (for tabular process data). The system should handle multi-page documents and identify the relevant sections related to a workflow without requiring the user to highlight or copy-paste content.

Edge Cases — Document Upload

- Document contains multiple distinct processes: agent identifies each and asks the user which one(s) to diagram, or offers to diagram all of them as separate outputs.
- Document is a scanned image (non-searchable PDF): agent attempts OCR-based extraction; if not feasible, it informs the user and asks them to provide a text version.
- Document is very long (50+ pages): agent identifies and extracts only the sections most likely to contain process/workflow descriptions, and informs the user of what it focused on.
- Document contains tables describing steps: agent correctly interprets tabular process documentation and converts it to a flow.
- Document is a meeting transcript: agent extracts decisions, action items, and described processes — distinguishing between what was discussed versus what was agreed as a workflow.
- Uploaded file is corrupted or unreadable: agent returns a clear, non-technical error message and prompts the user to re-upload.
- Document is in a format not supported: agent informs the user of supported formats clearly.

6.1.3 Voice Input

The agent should support voice input (via the platform's voice-to-text pipeline) so that users can verbally narrate a workflow without typing. This is particularly relevant for users coming out of a client meeting and wanting to immediately capture the discussed process. Voice input should be transcribed and treated identically to text input.

Edge Cases — Voice Input

- Transcription errors in domain-specific terms (e.g., 'EBITDA' transcribed incorrectly): agent applies a post-processing step or fuzzy match against known EY terminology before parsing.

- User pauses mid-narration: the system handles natural pauses without cutting off input prematurely.
- Multiple speakers on a recording: agent notes it may not attribute actions to correct actors and asks for confirmation.

6.1.4 Mixed Input

Users may combine inputs — for example, uploading a document and then clarifying or extending it via the chat interface. The agent must maintain context across these inputs within a session, treating them as a coherent whole unless the user explicitly starts a new diagram.

6.2 Diagram Generation

Once the agent has processed the input, it must generate a structured flowchart rendered in the right-hand canvas pane.

6.2.1 Internal Representation

Before rendering, the agent must construct a structured intermediate representation of the diagram — essentially a node-edge graph with typed nodes, labelled edges, and metadata (e.g., actor/swimlane assignments, conditions on edges). This representation serves as the data model that the rendering engine consumes and that the iterative editing system modifies. A well-defined JSON schema is strongly recommended.

This is critical because the iterative editing feature (Section 6.3) must modify specific parts of this graph without regenerating the whole, which is only possible if a stable intermediate representation exists.

6.2.2 Rendering Engine

The rendering engine should produce clean, professional diagrams using a standard diagramming library. Recommended options include Mermaid.js (for rapid text-to-diagram rendering), React Flow (for interactive canvas-based diagrams), or D3.js (for maximum custom rendering control).

The rendered diagram must conform to standard flowchart conventions: rectangles for process steps, diamonds for decision points, rounded rectangles or ovals for start/end terminators, parallelograms for inputs/outputs where appropriate, and labelled arrows for flow direction.

6.2.3 Layout & Styling

The default layout should be top-to-bottom for linear flows and left-to-right for horizontal process flows. Swim lanes should be clearly delineated with actor labels. Node labels should be concise (ideally under 10 words per node) — the agent should automatically summarise long step descriptions into short labels while preserving the full description as a tooltip or annotation.

The visual style should be clean and professional, consistent with what EY would include in a client presentation. A default EY-aligned colour scheme should be configurable.

Edge Cases — Generation

- Diagram is extremely large (50+ nodes): agent offers to split it into a high-level overview with drill-down sub-flows rather than rendering one unwieldy diagram.

- Circular/infinite loops in the process: agent detects cycles and explicitly labels them as loops; does not get stuck in an infinite generation state.
- Process has parallel branches that rejoin: agent correctly renders fork and join logic (AND split / AND join) distinct from XOR decision logic.
- Generated diagram does not match user's intent: user can provide corrective feedback (covered in Section 6.3). The system never silently overrides user feedback.
- Node label is ambiguous or empty due to extraction failure: agent renders a placeholder label (e.g., '[Step unclear — please clarify]') rather than omitting the node.

6.3 Iterative Editing — The Core Differentiator

This is the most important and differentiating feature of the agent, and must be specified with particular care. Once a diagram has been generated, the user must be able to modify it using natural language — and the modifications must be applied surgically to the affected portion of the diagram only, not by regenerating the entire diagram from scratch.

6.3.1 Surgical Edit Model

When a user provides an edit instruction (e.g., 'after the approval step, instead of going directly to payment, add a compliance check step first'), the agent must: (1) parse the instruction to identify which part of the existing diagram is being referenced; (2) locate the corresponding node(s) or edge(s) in the internal graph; (3) apply the modification to only those elements; and (4) re-render only the affected portion of the canvas while keeping the rest of the diagram intact.

This requires robust node identification. The user may reference nodes by their label, by their position ('the third step'), by their role ('the decision point after the review stage'), or by relative description ('the part where the manager gets involved'). The agent must resolve all of these reference types correctly.

Edge Cases — Iterative Editing

- User refers to a node ambiguously and multiple nodes match: agent asks the user to clarify which one they mean.
- User's edit would break the logical integrity of the diagram (e.g., removing a node that other edges depend on): agent warns the user about the downstream impact before applying the change.
- User asks to 'undo' the last edit: the system maintains an edit history stack and can revert to the previous state.
- User asks for a change that conflicts with an earlier constraint they set: agent flags the conflict and asks which constraint should take priority.
- User asks to split a segment: the system divides a subsection of the flow into a more detailed sub-flow and reintegrates it with correct entry and exit points.
- User asks to merge two steps into one: agent correctly combines their logic and updates all incoming and outgoing edges.
- User says 'start over': agent clears the current diagram and begins fresh, prompting for new input.
- User requests an edit that would require restructuring the entire diagram: agent informs the user this is a structural change (not surgical) and asks for confirmation before proceeding.

6.3.2 Edit History & Version Management

The agent must maintain a version history of the diagram within the session. Users should be able to: view a list of edits made during the session, undo individual edits (last-in-first-out), and restore a prior version by selecting from the history. This history need not persist across sessions in the initial version.

6.3.3 User-Initiated Branching

Advanced users may want to explore multiple versions of a diagram (e.g., 'show me what this looks like if we remove the manual approval step'). The system should support creating a named copy of the current diagram state so the user can explore an alternative without losing the original.

6.4 Output & Export

The agent must support exporting the generated diagram in multiple formats to enable use in client deliverables, presentations, and documentation.

Export Format	Description
PNG / JPEG	High-resolution image export for use in presentations, emails, and documents.
SVG	Scalable vector export for use in design tools and web embeds.
PDF	Print-ready export with professional formatting.
Mermaid / DOT syntax	Text-based export of the diagram definition for use with other rendering tools or version control.
Visio-compatible XML	For users who need to continue editing in Microsoft Visio.
JSON (internal graph)	For developers or power users who want the raw data model.

7. UI/UX Model — Split-Pane Interface

The UI model for this agent is a split-pane layout. The left pane contains the conversational interface (chat + input), and the right pane contains the live diagram canvas. Both panes must remain in sync at all times — any change to the diagram via chat is immediately reflected in the canvas, and (where applicable) direct manipulation of a node on the canvas should be reflected in the conversation log.

7.1 Left Pane — Conversational Interface

- Standard chat interface with a message input bar at the bottom.
- Supports text typing, file upload (drag-and-drop and file picker), and voice input toggle.

- The agent's responses in the chat should be concise and action-oriented — not verbose. When the agent makes an assumption, it states it clearly and briefly. When it asks a clarifying question, it asks only one at a time.
- The chat history scrolls upward and is persistent within the session.
- A 'Diagram History' panel allows users to view and restore previous versions of the diagram.
- Clear visual indicators when the agent is processing input (typing indicator, progress animation).

7.2 Right Pane — Diagram Canvas

- The canvas renders the live diagram and updates dynamically as edits are made.
- Users can pan and zoom the canvas to navigate large diagrams.
- Nodes are hoverable to reveal their full description/annotation where the label has been shortened.
- Nodes are selectable — clicking a node highlights it and enables direct editing via the chat (the node reference is auto-populated in the input bar).
- Export buttons are accessible from the canvas toolbar.
- A minimap is available for large diagrams to aid navigation.
- Undo/redo controls are available on the canvas toolbar.
- Swimlane labels (where applicable) are clearly rendered as column or row headers.

7.3 UX Principles

- Speed matters: the diagram should begin rendering as soon as the first structural elements are determined — progressive rendering is strongly preferred over waiting for full generation.
- Transparency: when the agent makes an assumption (e.g., infers a step that was not explicitly mentioned), it surfaces this assumption in the chat alongside the generated diagram.
- Forgiving: the agent never fails silently. If something goes wrong, the user always receives a clear, human-readable message explaining what happened and what they can do.
- Non-destructive: edits never permanently alter the diagram without the user being able to undo them. The edit history is the safety net.

8. Agent Workflow — End-to-End System Logic

This section describes the internal logic flow of the agent from user input to rendered output. This is the workflow the tech team should implement.

Step 1: Input Reception & Classification

When the user submits input (text, file, or voice), the system first classifies the input type and intent. For file uploads, it extracts text. For voice, it transcribes. The system then determines: is this a new diagram request, or a modification to an existing one? This classification drives which downstream pipeline is invoked.

Step 2: Content Extraction & Parsing

The LLM processes the input and extracts a structured representation of the described workflow. This includes identifying: the actors or roles involved, the sequential steps, decision points with their conditions and branches, parallel paths if any, start and end states, and the overall flow type. The output of this step is a structured JSON object representing the diagram's node-edge graph.

If the input is ambiguous or incomplete, the LLM generates a clarification question rather than proceeding with assumptions — unless the assumption is low-stakes and clearly reasonable (e.g., inferring a standard process start/end).

Step 3: Type Determination

Based on the extracted content, the system determines the most appropriate flowchart type (from Section 5). This selection can be overridden by the user. The selected type governs the rendering conventions applied in Step 4.

Step 4: Diagram Rendering

The internal JSON graph is passed to the rendering engine, which generates the visual diagram according to the conventions of the selected flowchart type. The rendered diagram appears in the right-hand canvas pane. A brief summary of what was generated and any assumptions made is posted to the chat.

Step 5: Iterative Edit Loop

The user reviews the diagram and either accepts it or provides edit instructions via the chat. If an edit is requested: the system parses the instruction, resolves the node reference(s) being targeted, modifies the internal JSON graph at the relevant node(s), and re-renders only the affected portion of the diagram. The edit is logged to the version history. This loop continues until the user is satisfied.

Step 6: Export

The user exports the final diagram in their preferred format via the canvas toolbar. The export is generated from the current state of the internal JSON graph and rendered to the selected format.

9. System-Level Edge Cases & Error Handling

In addition to the edge cases noted inline throughout Section 6, the following system-level scenarios must be handled.

Scenario	Expected Handling
Session timeout mid-edit	Warn user before timeout. On reconnect, restore the last saved diagram state from the session.
LLM returns malformed JSON graph	Backend must validate the JSON schema before passing to renderer. If invalid, retry up to 2 times with a corrected prompt before surfacing an error to the user.
Rendering library crashes or hangs	Catch error gracefully; show user a fallback static image or an error with retry option.
Extremely complex diagram (100+ nodes)	Proactively offer to decompose into a high-level diagram with linked sub-flows. Set a node limit threshold (e.g., 80 nodes) at which this prompt is triggered.
Duplicate nodes with same label	De-duplicate during parsing, or if intentional, append a numeric suffix to differentiate.
User pastes raw Mermaid or DOT syntax into chat	Detect and render it directly rather than treating it as natural language.
User requests unsupported diagram type	Inform user of supported types and suggest the closest match. Do not crash or return a blank diagram.
Model hallucinates a step not in input	Maintain an audit trail of extracted steps vs. input for debugging. Users can flag hallucinated steps, which should be logged for model improvement.

10. Technical Considerations

10.1 On-Premise Deployment

The agent runs entirely within EY's on-premise infrastructure. No data — user inputs, uploaded documents, or generated diagrams — should leave EY's network boundary. The LLM powering the agent is the proprietary model deployed by the platform provider. All API calls, rendering, and storage occur within EY's environment.

10.2 Internal Representation Schema

The engineering team should define and version a canonical JSON schema for the diagram graph object. This schema should support: typed nodes (process step, decision, start/end, swimlane, annotation), typed edges (default flow, conditional flow with label, bidirectional), metadata fields (actor, system, risk indicator, control indicator), and a version identifier. Defining this schema clearly upfront is essential for the iterative editing feature to function correctly.

10.3 LLM Prompting Strategy

The quality of diagram extraction is highly sensitive to prompt design. The engineering team should invest in structured prompting with few-shot examples for each flowchart type. Prompts should instruct the LLM to output only valid JSON in the defined schema and to explicitly flag missing or ambiguous information rather than guessing. Chain-of-thought prompting may improve accuracy for complex multi-branch processes.

10.4 Rendering Library Recommendation

React Flow is recommended as the primary rendering engine given its interactive canvas, node selection, zoom/pan support, and extensibility. Mermaid.js can serve as a fallback or an alternative export format. The rendering layer should be decoupled from the data model so the rendering engine can be swapped without affecting the agent's core logic.

10.5 Performance Targets

Target latency from input submission to first diagram render should be under 8 seconds for typical inputs (under 500 words or under 5 pages). Progressive rendering is strongly preferred over a single-shot render. Edit operations should reflect in the canvas within 3 seconds for targeted modifications.

11. Open Questions & Decisions Required

Open Question	Notes
Rendering Library	Final decision on React Flow vs. Mermaid.js vs. custom D3 renderer needed from tech lead.
Session Persistence	Should diagrams be saved across sessions? If so, the platform's storage layer needs to be scoped.
EY Visual Branding	Should the default diagram style match EY's brand guidelines exactly? Need design input from EY stakeholder.

Voice Input Pipeline	Is a voice-to-text module already available in the platform, or does it need to be built/integrated?
Supported File Types	Confirm whether PPTX and CSV extraction are in scope for v1, or deferred to a later version.
Node Limit Policy	What is the maximum number of nodes before the system auto-decomposes? Needs empirical testing.
Multi-language Support	Is non-English input a v1 requirement? EY operates globally — this may need explicit scoping.
Collaboration	Is single-user per session sufficient for v1, or does EY expect shared editing within a session?

12. Success Metrics

The following metrics should be tracked post-launch to evaluate the agent's effectiveness.

- Time to first diagram: average time from user submitting input to first rendered diagram. Target: under 8 seconds.
- Edit accuracy: percentage of edit instructions that are applied correctly to the intended node(s) without requiring a follow-up correction.
- Diagram completion rate: percentage of sessions where the user successfully exports a diagram, as a proxy for task completion.
- Clarification frequency: average number of clarifying questions the agent asks per session. A very high number suggests input processing is too uncertain; a very low number may suggest the agent is making too many assumptions.
- User satisfaction: collected via a brief post-session rating surfaced at the end of each diagramming session.
- Hallucination rate: percentage of generated steps that users flag as not present in their input. Tracked for model improvement.

13. Appendix

A. Flowchart Type Decision Matrix

The following table can guide the agent's automatic type selection when the user does not explicitly specify a type.

When User Input Contains...	Recommended Flowchart Type	Primary EY Service Line

User describes a sequence of tasks with roles	Process Flow (Swim Lane)	Consulting, Operations
User describes Yes/No conditions and outcomes	Decision Tree	Tax, Audit, Risk
User describes how data moves between systems	Data Flow Diagram	Tech Consulting, SaT
User describes who reports to whom	Org / Governance Flow	Consulting, HR
User describes risks and controls in a process	Risk & Control Flow	Audit, Assurance
User describes a customer's journey	Customer Journey Map	CX Consulting
User describes integration steps post-deal	Integration Workflow	Strategy & Transactions
User describes an approval or sign-off chain	Approval / Escalation Flow	Tax, Finance, Legal

— End of Document —